

Team Assignment 4 Question 1 - Maintainability & Component Organization

Admissions Alchemists

Joseph Rissman, Yue Zhang, Nitin Krishna Bojji

Feb 19, 2025

GradAid is designed with a strong emphasis on maintainability, achieved through a modular, component-based architecture. This approach not only enhances maintainability but also contributes to the system's scalability and flexibility. The system is composed of several key components, each with a specific responsibility. The frontend, built with Next.js, comprises several modules: the SOP/LOR Editor, providing a rich-text editing experience; the User Dashboard, displaying saved documents and application insights; Auth & Profile, handling user authentication and profile management; and the API Handler, facilitating communication with the backend via REST APIs. The backend, built with FastAPI, includes the SOP/LOR Generation API, leveraging LangChain for document generation; User Management, handling authentication, user data, and session tracking; and a Versioning Layer to ensure API compatibility. The Database, using Supabase/PostgreSQL, stores user data and generated documents. Finally, the LLM Engine, powered by LangChain, consists of Prompt Templates, AI Processing for interacting with LLMs, and embeddings for retrieval-augmented generation (RAG).

Managing dependencies and minimizing coupling between these components is crucial for maintainability. GradAid achieves this through several strategies. Frontend-Backend Decoupling is enforced by using REST APIs for communication. This allows for independent updates and even replacement of either the frontend or backend without affecting the other. A microservice-like approach isolates the LLM Engine (LangChain) from the core backend logic. This separation allows for experimentation with different AI models and libraries without impacting other backend functionalities. Database Abstraction, facilitated by Supabase/PostgreSQL, provides structured data storage and simplifies database migrations. Finally, a modular codebase within the backend, with functions grouped into modules like authentication, document generation, and user management, further reduces interdependencies and promotes code reusability.

GradAid employs a structured update strategy to ensure smooth transitions and minimal disruption. API Versioning is implemented, with each API endpoint following a versioning scheme (e.g., `/api/v1/generate_sop``). This allows older API versions to remain functional while new versions are developed, tested, and deployed, ensuring backward compatibility. A Plug-in Architecture for AI Models within the LangChain module allows for easy switching between different LLMs, such as OpenAI, local LLMs, or custom fine-tuned models. This modularity enables the addition of new AI features without affecting existing functionalities. Frontend updates are streamlined through Next.js deployments on Vercel, which also provides automatic rollbacks. Backend services are deployed using Docker and AWS Lambda for scalable updates. Database schema updates are managed using Alembic, a Python ORM, ensuring controlled and traceable database migrations.

A comprehensive Testing Strategy further supports maintainability. Unit Testing is performed for each service, including API endpoints and AI response validation. Integration Testing verifies the seamless communication between the frontend, backend, and AI models. End-to-End Testing simulates real-world user scenarios before each deployment, ensuring that the entire system functions correctly. This multi-layered testing approach helps catch bugs early in the development process, reducing the cost and effort of fixing them later. By implementing a modular, decoupled, and versioned architecture, GradAid ensures maintainability, easy scalability, and minimal disruption during updates. These strategies lay the foundation for seamlessly integrating future enhancements, such as a resume builder, application tracker, advanced AI features, and additional document types, over time.